Client Proposal

What I Built

I created the backend foundation for a matchmaking platform with NestJS and MongoDB. The system is composed of a personalized recommendation engine, swipe-based interactions (like/pass), mutual match detection, and a secure stateful session-based authentication system with opaque tokens as opposed to traditional JWT-only authentication.

The matchmaking engine uses weighted compatibility scoring to intelligently rank profiles based on shared interests, mutual age preferences and location similarity. It also tracks swipe history to prevent duplicate profile recommendations.

Enhancement

One of the major improvements made was a custom compatibility scoring algorithm paired with a modern stateful authentication setup. Rather than showing profiles randomly, the platform calculates compatibility using a scoring system that emphasizes shared interests through Jaccard similarity, matching mutual age preferences, and considering location relevance. This leads to a more personalized and meaningful recommendation system for users.

For authentication, instead of fully relying on self-contained JWTs, I introduced database-backed opaque session tokens. Each session token is randomly generated, hashed with SHA-256, and securely stored in the database with the associated user session. With every request, the incoming token is hashed and checked against the stored session record before processing the request. This method boosts security by allowing immediate session revocation, safer handling of token leaks, active session management, and device-level authentication control while keeping the actual token payload opaque and unreadable.

Next Steps

- Implement profile image uploads using Amazon S3 for scalable and secure media storage
- Add real-time chat and notifications using WebSockets
- Write comprehensive unit, integration, and end-to-end test cases for authentication and matchmaking flows

- Add geospatial matching using latitude/longitude
- Perform load and stress testing to evaluate scalability and request handling under high traffic
- Introduce AI-based recommendation improvements using behavioral and engagement signals
- Implement a reporting and moderation system with admin roles, dashboards, and user ban capabilities for platform safety

---

Technical Approach

The platform has a modular backend architecture built with NestJS, MongoDB, and a session authentication system that maintains state. User authentication uses opaque session tokens, which are stored securely in the database. Match actions, like likes and passes, are tracked separately. This setup allows the recommendation engine to leave out previously viewed profiles and calculate compatibility scores dynamically. When there are mutual likes, it triggers logic to create a match, which can later support messaging and real-time interactions.

Simple Architecture Diagram:

Client Application
↓
HTTP-only Session Token
↓
NestJS API Server
↓
SHA-256 Token Hash Validation
↓
MongoDB Session Store
↓
Recommendation Engine
↓
Compatibility Scoring + Match Actions
↓
Mutual Match Detection
↓
Future Real-Time Chat System